Declaration of Original Work for SC2002/CE2002/CZ2002 Assignment

We hereby declare that the attached group assignment has been researched, undertaken, completed, and submitted as a collective effort by the group members listed below.

We have honored the principles of academic integrity and have upheld Student Code of Academic Conduct in the completion of this work.

We understand that if plagiarism is found in the assignment, then lower marks or no marks will beawarded for the assessed work. In addition, disciplinary actions may be taken.

Name	Course (SC2002/CE2002 CZ2002)	Lab Group	Signature /Date
Joel Tan Qin Xuan	SC2002	SCMA	Joel /19-11-24
Lee Lin Yang Glenn	SC2002	SCMA	Glenn /19-11-24
Ong Jing Jie	SC2002	SCMA	JingJie /19-11-24
Shao Yingzhan	SC2002	SCMA	YingZhan /19-11-24
Lau Kai Feng	SC2002	SCMA	KaiFeng /19-11-24

ADDITIONAL FEATURES/FUNCTIONALITIES

1. Password Encryption:

We are aware that there will be security risks if we were to store all passwords without encrypting. Hence, we made use of SHA-256 algorithm which is a cryptographic hash function used to securely encrypt data by converting it into a unique 256-bit hash. Commonly employed in cybersecurity, blockchain, and data integrity verification, it generates hashes that are challenging to reverse, ensuring data security. Due to its robustness, SHA-256 remains a trusted standard in secure communication protocols.

2. MVC

The Model-View-Controller (MVC) architecture separates an application into three interconnected components. The *Model* manages data and business logic, the *View* displays data and sends user commands, and the *Controller* acts as an intermediary, processing input and updating the Model and View. This separation enhances modularity, making applications easier to develop, test, and maintain.

3. Appointment ID Generation

To streamline appointment management by reducing appointment ID lengths and ensure each record is uniquely identifiable, we implemented a custom ID generation method. The algorithm combines a timestamp and a counter to create compact and human-readable IDs. Specifically, we use the last 5 hexadecimal digits of the current system timestamp and append a single hexadecimal digit derived from a cyclic counter. This approach minimizes the length of IDs while maintaining uniqueness and scalability. By resetting the counter cyclically, we avoid excessive growth in ID size without sacrificing uniqueness during a single timestamp. This method enhances readability and efficiency compared to longer IDs solely based on timestamps.

DESIGN CONSIDERATIONS (SOLID principles)

1. Single Responsibility Principle (SRP)

The main idea of SRP is that a class should only exist just for a single reason/responsibility. This can be seen from our implementation of "Authenticator", which exists just to check if an user who is trying to log in into our hospital management system is authenticated.

2. Open-Closed Principle (OCP)

We have a class "Staff" which extends from an abstract super class "User". Staff is an extensible class whereby if the developer chooses to add more different kinds of staff roles for example, cleaning staffs or maintenance crews, he/she can extend from this Staff class in order to create the different subclasses.

3. Liskov Substitution Principle (LSP)

LSP says that subclasses should be substitutable for its base class if pre-conditions are no stronger than the base class method (expect no more) and its post-conditions are no weaker than the base class method (provide no less). This can be seen from the UserView class that implements the ViewInterface, which defines a single method, displayMenu(). This ensures that UserView can be used in any context where ViewInterface is expected without breaking the program's behavior. The displayMenu() method in UserView aligns with the contract defined by the ViewInterface, making it LSP-compliant.

4. Interface Segregation Principle (ISP)

Our DataReader and DataWriter are examples of the ISP principle. The DataWriter Interface is designed around writing data to CSV files. It has the methods saveRecords(), createHeader(), and createCells() which are directly related to saving a specific type of object

(like Patient, Medicine, etc.). This interface is segregated properly since it is focused on one task: writing objects to CSV. A class that implements DataWriter<Patient> only needs to implement methods that are relevant to saving Patient objects. The DataReader interface is responsible for reading data from CSV files and populating objects (e.g., Patient, Medicine). There is only 1 method, populateData(DataStorage dataStorage). Having a separate interface for reading and writing prevents the classes from being forced to implement a method that they do not need.

5. Dependency Injection Principle (DIP)

DIP states that both high and low level modules should depend on abstractions or interfaces. This can be seen in the interface, ViewInterface, which is implemented by many other low-level modules such as PatientView and DoctorView. It is also depended upon indirectly by the high level module HospitalManagementSystem which runs the whole entire system, making use of UserView.

REFLECTION

Difficulties

1. How to R/W to CSV files without external libraries
    a. Reading CSV files in Java proved to be a challenging yet rewarding experience. Initially, I used Apache POI libraries, which required me to learn how to import dependencies despite limited guidance online. This was my first experience sourcing APIs independently, and it taught me the importance of reading and understanding documentation—a habit I hadn't fully appreciated before. This experience made it clear how valuable thorough documentation review can be. The next step was designing the classes to handle reading and writing to the CSV files effectively. We explored various update methods, such as modifying a single cell by providing the row, column, and new value or updating by specifying the userID, column, and input. At first, we considered an approach that would update the CSV immediately whenever a change occurred, such as when a password was modified. While this would provide high reliability by saving changes instantly, we recognized that it added excessive complexity. Instead, we opted for a simpler solution: updating the CSV only when the user logs out. This captures the current state of all

entities at once and writes it to the CSV, maintaining simplicity and ensuring that updates are consolidated efficiently. This approach led to more streamlined and manageable code.

2. How to separate the packages
    a. With the creation of many classes due to MVC model. There were many classes in the default package making code reading or finding a specific class quite difficult due to the sheer amount of classes in the default package. We recognized this issue but we wasn't sure how we wanted to group the classes up. We could have group based on entity or functionality however, we finally decided to separate the class based on what the class is about. For example, all controller-related classes are placed into the controller package. This approach led us to manage the code easier as it was significantly easier to find the classes that we were looking for. It also made the amount of classes in a package lower which adds to code management. Thus, separating the Hospital Management System (HMS) into packages was challenging but essential for clarity and maintainability.

Challenges

1. **Determining Structure**: Deciding on logical groupings for classes took time and required balancing functionality with separation of concerns.
2. **Managing Dependencies**: Cross-package dependencies were difficult to handle, especially for avoiding circular dependencies and ensuring data flow.
3. **Debugging**: Isolating issues across multiple layers added complexity to debugging and required careful testing.

Solutions

1. **Layered Architecture**: We followed a clear separation of responsibilities—`entities` for data, `controllers` for logic, `viewers` for UI, and `DataReadWrite` for data persistence—making each part modular.
2. **Dependency Injection**: Passing dependencies through constructors reduced coupling and improved flexibility.

3. **Centralized Data Handling**: The `DataReadWrite` package managed data access, ensuring consistency and simplifying modifications.

This structure made the HMS easier to manage, test, and extend, despite the initial challenges.

Furthermore improvement suggestions

1. **Pharmacist view could be used to track down history of patients**
2. **Appointment ID can be continuous**
3. **Patient is able to view history of appointment / canceled appointment**
4. **Immediate saving(saving whenever there is changed) instead of saving at intervals(Saving only when user logs out). Immediate saving will improve reliability.**
5. **Using a static variable to hold the data for controllers to avoid passing in the DataStorage everytime the controller is needed to be created.**

1. Pharmacist View for Tracking Patient History:

The Pharmacist View makes it easy for pharmacists to look up a patient's history, including past prescriptions and any medical advice given. This helps pharmacists provide better care by understanding the patient’s needs and history at a glance. It also saves time and reduces mistakes, making the process smoother and more efficient.

2. Continuous Appointment ID Generation:

Our system ensures that every appointment gets a unique and continuous ID, even if it’s canceled or rescheduled. This makes it easier to keep track of all appointments in order and simplifies reviewing or auditing appointment records.

3. Patient Access to Appointment History:

Patients can easily check their appointment history, including both attended and canceled appointments. This feature keeps everything transparent and helps patients stay on top of their healthcare by having all their past appointments in one place.

4. Immediate Saving for Improved Reliability:

The system saves changes as soon as they happen, so we don't have to worry about losing updates if something goes wrong, like in the case of a power outage. This real-time saving also ensures the latest data is always available, which is essential if we need to support many concurrent users making the system more reliable and user-friendly.

5. Static variable to hold Data Records:

All the controllers can be created initially with a constructor taking in DataStorage to save Data Records to a static variable for the controller class. This is so that subsequent creation of the controller will not require to pass in DataStorage into the constructor while still being able to use the static variable for data adjustment. This is so that data can be more encapsulated.

TESTING

1. Patient Actions

Test case 1: View Medical Record

```
Patient ID: P1001
Patient Name: Alice Brown
Patient Gender: Female
Patient Date Of Birth: 14/5/1980
Patient Email: AliceBrown@gmail.com
Patient Contact Number: 61234567
Patient Blood Type: A+
Patient Diagnoses: NULL
Patient Treatment: NULL
```

Test case 2: Update Personal Information

```
------ Update Personal Information ------
1. Email Address
2. Contact Information
3. Exit
1
Enter New Email Address
AliceBrown123@gmail.com
Email Address Updated
```

```
------ Update Personal Information ------
1. Email Address
2. Contact Information
3. Exit
2
Enter New Contact Information
98765432
Contact Information Updated
```

Test case 3: View Available Appointment Slots

```
------ Available Appointment Slots ------
Appointment ID: AP0001, Doctor ID: D001, Date: 11/13/2024, Time: 13:00, Status: AVAILABLE
Appointment ID: AP0005, Doctor ID: D001, Date: 10/10/2024, Time: 11:00, Status: AVAILABLE
Appointment ID: AP0008, Doctor ID: D001, Date: 10/10/2024, Time: 10:00, Status: AVAILABLE
Appointment ID: AP0011, Doctor ID: D001, Date: 12/12/2024, Time: 11:00, Status: AVAILABLE
Appointment ID: AP0012, Doctor ID: D001, Date: 10/10/2024, Time: 12:00, Status: AVAILABLE
```

Test case 4: Schedule an Appointment

```
------ Schedule an Appointment ------
Enter Doctor ID: D001
Enter Appointment Date (DD/MM/YYYY): 10/10/2024
Enter Appointment Time (HH:MM): 11:00
Appointment No: AP0005 is now pending.
```

Test case 5: Reschedule an Appointment

```
------ Reschedule an Appointment ------
Enter Appointment ID to reschedule: AP0002
Enter New Appointment Date (DD/MM/YYYY): 10/10/2024
Enter New Appointment Time (HH:MM): 12:00
Appointment No: AP0002 rescheduled successfully.
New Appointment No: AP0012 is now pending.
```

Test case 6: Cancel an Appointment

```
------ Cancel an Appointment ------
Enter Appointment ID to cancel: AP0012
Appointment canceled successfully.
```

Test case 7: View scheduled Appointments

```
------ Scheduled Appointments ------
Appointment ID: AP0005, Doctor ID: D001, Date: 10/10/2024, Time: 11:00, Status: SCHEDULED
```

Test case 8: View Past Appointment Outcome Records

```
------ Past Appointment Outcome Records ------
Appointment ID: AP0003, Date: 13/11/2024, Time: 13:00, Status: COMPLETED, Prescribed Medicine: Paracetamol:10, Consultation Notes: Ok for now
Appointment ID: AP0004, Date: 11/13/2024, Time: 13:00, Status: COMPLETED, Prescribed Medicine: Paracetamol:20, Amoxicillin:20, Consultation No
```

2. Doctor Actions

Test case 9: View Patient Medical Records

```
Doctor Menu:
(1) View Patient Medical Records
(2) Update Patient Medical Records
(3) View Personal Schedule
(4) Set Availability for Appointments
(5) Accept or Decline Appointment Requests
(6) View Upcoming Appointments
(7) Record Appointment Outcome
(8) Logout
1
Name: Alice Brown
Blood Type: A+
DOB: 5/14/1980
Gender: Female
Diagnoses: 1
Treatment: 3
```

Test case 10: Update Patient Medical Records

```
------ Update Patient Medical Records ------
Enter Patient ID to update records: P1001
Enter new diagnoses: Found risk of diabetes
Enter new prescriptions: Insulin
Enter treatment plan: Medications
Patient records updated successfully.
```

Test case 11: View Personal Schedule

```
------ Doctor's Personal Schedule ------
----------------------------
Appointment ID: AP0005
Date & Time: 10/10/2024 11:00
Patient ID: P1001
----------------------------
```

Test case 12: Set Availability for Appointments

```
------ Set Availability for Appointments ------
Enter date for availability (DD/MM/YYYY): 30/11/2024
Slots available are:
[09:00, 10:00, 11:00, 12:00, 13:00, 14:00, 15:00, 16:00, 17:00]
Enter available time slot (HH:MM) (e.g., 09:00): 16:00
Availability set successfully.
Do you want to set another time slot? (yes/no): no
```

Test case 13: Accept or Decline Appointment Requests

```
------ Accept/Decline Appointment Requests ------
Appointment ID: AP0009, Date: 12/12/2024, Time: 9:00, Patient ID: P1001
Appointment ID: AP0010, Date: 12/12/2024, Time: 10:00, Patient ID: P1001
Enter Appointment ID to Accept/Decline: AP0009
Accept (A) or Decline (D): A
Request processed successfully.
```

```
------ Accept/Decline Appointment Requests ------
Appointment ID: AP0010, Date: 12/12/2024, Time: 10:00, Patient ID: P1001
Enter Appointment ID to Accept/Decline: AP0010
Accept (A) or Decline (D): D
Request processed successfully.
```

Test case 14: View Upcoming Appointments

```
------ Upcoming Appointments ------
Appointment ID: AP0005, Date: 10/10/2024, Time: 11:00, Patient ID: P1001
Appointment ID: AP0009, Date: 12/12/2024, Time: 9:00, Patient ID: P1001
```

Test case 15: Record Appointment Outcome

```
------ Record Appointment Outcome ------
Enter Appointment ID: AP0005
Enter Date of Appointment (DD/MM/YYYY): 10/10/2024
Enter Type of Service (e.g., Consultation, X-ray): Consultation
Enter any Prescribed Medications (comma-separated):
Enter Medications QTY (comma-separated), Enter 0 if none: 0
Enter Consultation Notes: Need to revisit again
Outcome recorded successfully.
```

3. Pharmacist Actions

Test case 16: View Appointment Outcome Record

```
------ Past Appointment Outcome Records ------
1. AP0003
2. AP0004
3. AP0005
4. Return to Menu
1
----- Appointment Details -----
Appointment ID: AP0003
Doctor ID: D001
Patient ID: P1001
Appointment Date: 13/11/2024
Prescribed Medicines:
Paracetamol | Quantity: 10 | Status: Pending
Appointment Type: Consultation
Consultation Notes: Ok for now
-------------------------------
```

Test case 17: Update Prescription Status

```
------ Update Prescription Status ------
Enter Appointment ID: AP0003
Paracetamol remaining stock :  120
Medicine for AP0003 has been successfully dispensed.
```

Test case 18: View Medication Inventory

```
------ Inventory -------
1 Paracetamol       |  Stock :  120  |  Stock Alert : 20
2 Ibuprofen         |  Stock :   50  |  Stock Alert : 10
3 Amoxicillin       |  Stock :   65  |  Stock Alert : 15
```

Test case 19: Submit Replenishment Request

```
------- Medicine Replenishment -------
1. Paracetamol
2. Ibuprofen
3. Amoxicillin
4. Return to Menu
Select Option: 2
To Cancel Replenishment Enter Values LESS OR EQUAL TO 0
Enter Quantity to Replenish: 50
Replenishment Request Submitted... Returning to menu...
```

4. Administrator Actions

Test case 20: View and Manage Hospital Staff

```
------ List of Hospital Staff ------
UserID: D001 Name: John Smith Role: Doctor Gender: Male Age: 45
UserID: D002 Name: Emily Clarke Role: Doctor Gender: Female Age: 38
UserID: P001 Name: Mark Lee Role: Pharmacist Gender: Female Age: 40
UserID: A001 Name: Sarah Lee Role: Administrator Gender: Female Age: 40
```

```
------ Add Staff ------
Enter New Staff ID
A002
Enter Name
Max Goh
Enter Gender
Male
Enter Age
35
```

```
------ Remove Staff ------
Enter Staff ID
A002
```

Test case 21: View Appointments Details

```
------ Past Appointment Outcome Records ------
1. AP0001
2. AP0002
3. AP0003
4. AP0004
5. AP0005
6. AP0006
7. AP0007
8. AP0008
9. AP0009
10. AP0010
11. AP0011
12. AP0012
13. AP0013
14. AP0014
15. Return to Menu
Enter Option: 1
----- Appoint Details -----
Appointment ID: AP0001
Date & Time : 11/13/2024 13:00
Appointment Status: AVAILABLE
Doctor ID: D001
Patient ID: NULL
Prescripted Medicines:
-------------------------
```

Test case 22: View and Manage Medication Inventory

```
------ Inventory Management -------
1. View Inventory
2. Manage Inventory
3. Return To Menu
Enter Option: 1
------ Inventory -------
1 Paracetamol      |  Stock :  120  |  Stock Alert : 20
2 Ibuprofen        |  Stock :   50  |  Stock Alert : 10
3 Amoxicillin      |  Stock :   65  |  Stock Alert : 15
```

```
------ Inventory Management -------
1. View Inventory
2. Manage Inventory
3. Return To Menu
Enter Option: 2
1. Adding stock levels
2. Removing stock levels
3. Updating Low stock alert
4. Return To Inventory Management
Enter Option: 1
Enter Medicine Name to Add Stock: Amoxicillin
Enter Quantity to Add: 15
Amoxicillin stock adjusted to 80
```

Test case 23: Approve Replenishment Requests

```
---- Pending Replenishment Requests ----
1. Amoxicillin | Quantity: 40
2. Ibuprofen | Quantity: 50
3. Exit
Select Replenishment To Approve or Exit: 2
Ibuprofen stock replenished to 100
```

5. Login System and Password Management

Test case 25: First-Time Login and Password Change

```
--- Welcome to Hospital Management System ---
1. Login
2. Exit
Choice: 1
Enter User ID:  D003
Enter Password: password
Login Successful...
You are using the default password.
You are required to change your password.
Enter New Password:
```

```
Password has been updated.
Doctor Menu:
(1) View Patient Medical Records
(2) Update Patient Medical Records
(3) View Personal Schedule
(4) Set Availability for Appointments
(5) Accept or Decline Appointment Requests
(6) View Upcoming Appointments
(7) Record Appointment Outcome
(8) Logout
```

Test case 26: Login with Incorrect Credentials

```
--- Welcome to Hospital Management System ---
1. Login
2. Exit
Choice: 1
Enter User ID:  P1001
Enter Password: testtest
Invalid Credentials. Please Try Again...
Enter User ID:
```